

Step 3 — Add the Resource and the Prompt — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# db.py (new function)
def list_all_notes(path):
    conn = sqlite3.connect(path)
    rows = conn.execute("SELECT id, text, created_at FROM notes ORDER BY id").fetchall()
    conn.close()
    return rows
```

```
# server.py (new pieces)
@mcp.resource("notes://all")
def all_notes():
    rows = list_all_notes(DB_PATH)
    lines = [f"[{row[0]] {row[1]}" for row in rows]
    return "\n".join(lines)

@mcp.prompt()
def summarize_notes(start_date, end_date):
    return f"Summarize the notes created between {start_date} and {end_date}."
```

This works for a non-empty store. It has no type hints, no docstrings (so a client's UI has nothing to show the user about what each one does), and returns an empty string instead of a clear message when there are no notes yet — which looks like a bug to anyone reading the result, not like "correctly reporting there's nothing here."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, documented, and an explicit empty-store message

```
# db.py (new function)
def list_all_notes(path: str) -> list[tuple[int, str, str]]:
    conn = sqlite3.connect(path)
    rows = conn.execute("SELECT id, text, created_at FROM notes ORDER BY id").fetchall()
    conn.close()
    return rows
```

```
# server.py (new pieces)
from db import list_all_notes

@mcp.resource("notes://all")
def all_notes() -> str:
    """A read-only listing of every note currently stored, oldest first."""
    rows = list_all_notes(DB_PATH)
    if not rows:
        return "No notes yet."
    lines = [f"[{row[0]}] {row[1]} ({row[2]})" for row in rows]
    return "\n".join(lines)

@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
    """A reusable template for summarizing notes created in a date range
    (format: YYYY-MM-DD). Returns the prompt text only -- it does not call
    a model or fetch notes itself."""
    return (
        f"Summarize the notes created between {start_date} and {end_date} "
        "in 2-3 sentences. Group related notes by theme if there's more than one."
    )
```

Expected behavior: reading `notes://all` on a fresh database returns `"No notes yet."`. After adding "Buy milk and eggs" (id 1) and "Call the dentist to reschedule" (id 2), it returns:

```
[1] Buy milk and eggs (2026-09-11T10:03:12.481022)
[2] Call the dentist to reschedule (2026-09-11T10:04:01.998710)
```

Calling `summarize_notes` with `start_date="2026-09-01"`, `end_date="2026-09-30"` returns the filled-in template string, unchanged by how many notes actually exist — it's a template, not a report.

Difference from Basic: full type hints and real docstrings (a Resource's and a Prompt's descriptions matter to a client's UI the exact same way a Tool's description matters to a model — Doc06's "a tool's description is really a prompt" idea, one level more general than just Tools), and an explicit `"No notes yet."` message instead of a silent empty string.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the Prompt stays a pure template (recommended)

```
@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
    """A reusable template for summarizing notes created in a date range
    (format: YYYY-MM-DD). Returns the prompt text only -- pair this with
    the notes://all Resource in the same conversation for the client to
    actually have the notes' content available to summarize."""
    return (
        f"Using the attached notes (see notes://all), summarize the ones "
        f"created between {start_date} and {end_date} in 2-3 sentences. "
        "Group related notes by theme if there's more than one."
    )
```

Expected behavior: identical shape to Intermediate — the function still never touches the database. The only change is the wording, which now explicitly tells whoever uses this Prompt to attach `notes://all` alongside it.

Approach 2 — the Prompt embeds matching notes directly (why this is worse here)

```
@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
    rows = list_all_notes(DB_PATH)
    matching = [r for r in rows if start_date <= r[2][:10] <= end_date]
    notes_text = "\n".join(f"- {r[1]}" for r in matching)
    return f"Summarize these notes from {start_date} to {end_date}:\n{notes_text}"
```

Expected behavior: this also works, and even saves the client one step (no need to attach `notes://all` separately). But it now duplicates `list_all_notes()`'s job inside a Prompt function, means the Prompt's output changes every time the underlying data changes (a Prompt *template* is supposed to be stable; the data behind it isn't), and blurs a line this whole step exists to teach you to keep sharp — a Prompt building its own private read path around the Resource that already exists for exactly this.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate already keeps Prompt and Resource cleanly separated; Approach 1 only sharpens the wording to

make that separation obvious to whoever reads it. Approach 2 is a real, working alternative — not a mistake to catch, a genuine design choice — that trades a clean Tool/Resource/Prompt boundary for one fewer step in the client's workflow.

Which one should you actually write? Approach 1. The whole point of this step is telling Tools, Resources, and Prompts apart by what they're *for*, not just making the final output convenient. A Prompt that quietly re-implements a Resource's job the moment it's convenient is the same mistake as a Tool with a vague description (Doc06's opening lesson) wearing a different costume — it works today, and it's the first thing that gets confusing once this server has more than one Resource and more than one Prompt, and someone has to figure out which one actually reads the database.